

Trinity Dropbox Scheduler - Process and File Guide

Generated: May 15, 2026

Project path: /Users/anand/Projects/trinity-dropbox-scheduler

Purpose: This project watches a Dropbox app folder for incoming PDF authorizations, extracts appointment and claimant data, fills a Word report template, updates a same-day Word schedule, uploads the schedule to Dropbox, and moves the source PDF to a processed folder.

Setup Document Review

Verdict: The existing "scheduler local enabler steps.docx" is mostly correct for installing the project on a Mac, creating a virtual environment, setting up Dropbox API credentials, creating Dropbox folders, testing manually, and installing the LaunchAgent background job.

Corrections needed because the code has changed:

- The .env section must now also include Gmail SMTP alert settings: SMTP_HOST, SMTP_PORT, SMTP_USE_TLS, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM_EMAIL, and ALERT_TO_EMAIL.
- The config/settings.json example should include the health_check block with log_folder, email_notifications_enabled, and email_subject_prefix.
- The manual test section should mention that failures are written to logs/scheduler_error.log and, when SMTP is configured, an alert email is sent.
- requirements.txt contains python-docx twice. This does not break setup, but it can be cleaned up later.
- app/state_manager.py is present but not used by the current main workflow. This does not break the scheduler.

Runtime Flow

1. run_scheduler.sh is called manually or by macOS launchd.
2. run_scheduler.sh changes into the project folder, activates venv, and runs python main.py.
3. main.py loads config/settings.json and sets up rotating logs.
4. Startup health checks confirm required config values, template paths, and the generated report folder.
5. The Dropbox API lists PDF files from /incoming_pdfs.
6. Each PDF is downloaded into data/incoming_pdfs.
7. app/pdf_extractor.py reads PDF text and extracts case ID, authorization, patient name, doctor, examiner initials, DOB, phone, services, total, allegations, schedule date, and schedule time.
8. app/template_filler.py chooses the adult or child report template and replaces Word placeholders.
9. app/schedule_generator.py creates or updates Schedule_<date>.docx with a 9-column table, including an empty Comments column.
10. The schedule DOCX is uploaded to Dropbox /schedules.
11. The source PDF is moved from /incoming_pdfs to /processed_pdfs.
12. If one PDF fails, the scheduler logs the full traceback, emails an alert if enabled, and continues to the next PDF.
13. If the whole run fails outside a single PDF, main.py logs a fatal error and sends a fatal scheduler failure email.

Dropbox Layout

The app is configured for a Dropbox App Folder style app. Inside Dropbox/Apps/TrinityScheduler, create:

- incoming_pdfs: staff drop new PDFs here.
- processed_pdfs: successfully processed PDFs are moved here.
- schedules: generated or updated schedule DOCX files are uploaded here.

The config paths are relative to the app folder: /incoming_pdfs, /processed_pdfs, and /schedules.

Environment Variables

Required Dropbox values in .env:

```
DROPBOX_APP_KEY=<Dropbox app key>
DROPBOX_APP_SECRET=<Dropbox app secret>
DROPBOX_REFRESH_TOKEN=<offline refresh token from get_refresh_token.py>
```

Required Gmail alert values when email_notifications_enabled is true:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USE_TLS=true
SMTP_USERNAME=<sending Gmail address>
SMTP_PASSWORD=<Google app password, not the normal Gmail password>
SMTP_FROM_EMAIL=<same sending Gmail address>
ALERT_TO_EMAIL=<alert recipient email address>
```

Configuration File

config/settings.json controls local folders, Dropbox folders, template paths, batch size, adult cutoff age, and health check behavior.

Current important values:

- base_folder: data
- dropbox_input_folder: /incoming_pdfs
- dropbox_processed_pdf_folder: /processed_pdfs
- dropbox_schedule_folder: /schedules
- adult_age_cutoff: 18
- max_files_per_run: 200
- adult_template_path: templates/adult_report_template.docx
- child_template_path: templates/child_report_template.docx
- generated_docs_folder: /Users/anand/Desktop/reports
- health_check.log_folder: logs
- health_check.email_notifications_enabled: true

File Map

main.py: Main orchestrator. Loads config, creates/removes scheduler.lock, runs health checks, loops through Dropbox PDFs, calls extraction/template/schedule/upload/move steps, and handles error notifications.

run_scheduler.sh: Shell wrapper for background or manual execution. Activates venv and runs main.py while appending stdout/stderr to logs.

config/settings.json: Runtime configuration for Dropbox folders, local folders, templates, max files per run, adult age cutoff, and health monitoring.

.env: Secret local configuration. Holds Dropbox credentials and Gmail SMTP alert

credentials. This file should not be zipped or committed.

.env.example: Non-secret template showing which environment variables are expected.

requirements.txt: Python dependencies: python-dotenv, dropbox, pdfplumber, python-docx, and python-dateutil. python-docx is duplicated and can be cleaned later.

get_refresh_token.py: One-time helper that opens a Dropbox authorization flow and prints DROPBOX_REFRESH_TOKEN for .env.

apis/dropbox_api.py: Dropbox wrapper functions for auth, listing PDFs, downloading files, uploading schedules, checking for existing schedules, and moving processed PDFs.

app/config_loader.py: Reads config/settings.json.

app/pdf_extractor.py: Uses pdfplumber and regexes to parse authorization PDFs and return a structured dictionary used by the rest of the app.

app/template_filler.py: Opens adult or child Word report templates, replaces placeholders in paragraphs, tables, headers, and footers, formats DOB/age/date fields, and writes the generated report DOCX.

app/schedule_generator.py: Creates or updates the same-day schedule DOCX. It sets landscape layout, formats the table, avoids duplicate case IDs, sorts rows by date/time, and includes a blank Comments column.

app/health_monitor.py: Sets up rotating logs, performs startup checks, builds traceback email bodies, and sends Gmail SMTP alerts.

app/state_manager.py: Legacy helper for tracking processed Dropbox file IDs. It is not currently used by main.py.

app/file_utils.py: Empty placeholder file; not used by the current workflow.

Word Template Placeholders

The report templates should contain these placeholders where needed:

{{FULL_NAME}}

{{FIRST_NAME}}

{{LAST_NAME}}

{{CASE_NUMBER}}

{{DOB}}

{{DATE_OF_EXAMINATION}}

{{REFERRED_BY}}

{{ALLEGATIONS}}

{{EXAMINER}}

{{DOCTOR_FULL_NAME}}

The code preserves template formatting by replacing text inside existing Word runs. EXAMINER and DOCTOR_FULL_NAME are forced red when replaced.

Schedule DOCX Details

Schedule file name format: Schedule_<Month_Day_Year>.docx, for example

Schedule_May_10_2026.docx.

Schedule columns: Case ID, Patient / Doctor / Phone, DOB, Adult, Child, Date & Time, Services Requested, Allegations, Comments.

The Comments column is intentionally blank for every new row.

If an older 8-column schedule is downloaded, schedule_generator.py upgrades it to the current 9-column format before saving.

Rows are sorted using the Date & Time column. Unknown or unparsable dates sort last.

Logging and Alerts

Normal activity is written to logs/scheduler.log.

Errors and tracebacks are written to logs/scheduler_error.log.

Log files rotate at about 2 MB, keeping 5 backups.

When a single PDF fails, the app sends an email with the PDF name, timestamp, host, working directory, and traceback, then continues processing other PDFs.

When a fatal startup or scheduler-level failure occurs, the app sends a fatal scheduler failure email and exits with an error.

A successful test was performed with a deliberately broken PDF; the system emailed a PdfminerException traceback as expected.

Manual Test Checklist

1. Activate the environment: `source venv/bin/activate`
2. Run syntax check: `python -m py_compile main.py app/*.py apis/*.py get_refresh_token.py`
3. Test Gmail SMTP using `app.health_monitor.send_email`.
4. Put a valid test PDF in `Dropbox/Apps/TrinityScheduler/incoming_pdfs`.
5. Run `python main.py`.
6. Confirm a generated report appears in `generated_docs_folder`.
7. Confirm a schedule appears or updates in `Dropbox /schedules`.
8. Confirm the source PDF moves to `Dropbox /processed_pdfs`.
9. Test error alerts by putting a fake .pdf file in `/incoming_pdfs` and running `python main.py`.
10. Remove the fake PDF after testing so alerts do not repeat.

Background LaunchAgent Notes

The setup document LaunchAgent plist is structurally correct for running the scheduler on a Mac every 300 seconds.

The ProgramArguments path and WorkingDirectory must match the actual username and project path.

After changing the plist, unload and reload it with `launchctl`.

Use `launchctl list | grep com.trinity.dropbox.scheduler` to confirm it is loaded.

Use `tail -f logs/scheduler.log` and `tail -f logs/scheduler_error.log` to watch activity.

Known Cleanup Items

These are not blockers:

- Remove the duplicate `python-docx` line from `requirements.txt`.
- Consider converting remaining print calls in helper modules to logger calls for cleaner logs.
- `app/state_manager.py` and `app/file_utils.py` can be removed later if no future workflow needs them.
- `run_scheduler.sh` appends to the same log files that Python now writes to. This is acceptable, but the shell wrapper could be simplified later.